Neural Networks

Prerequisites :

Before diving into neural networks, it is essential to have a strong foundation in the following areas:

1. **Mathematics**
    - *Linear Algebra*: Understand vectors, matrices, and their operations.
    - *Calculus*: Grasp derivatives, gradients, and the chain rule.
    - *Probability & Statistics*: Know basic probability, distributions, and statistical measures.

      https://www.youtube.com/playlist?list=PLZHQObOWTQDMsr9K-rj53DwVRMYO3t5Yr

      https://www.khanacademy.org/math/linear-algebra

      https://www.youtube.com/watch?v=IHZwWFHWa-w&ab_channel=3Blue1Brown

2. **Programming Skills**
    - Proficiency in Python
    - Familiarity with libraries like NumPy, Pandas, and Matplotlib
    - Understanding object-oriented programming concepts

      https://realpython.com/python3-object-oriented-programming/

3. **Data Handling**
    - Techniques for cleaning, normalizing, and splitting datasets

      https://www.kaggle.com/learn/data-cleaning

4. **Machine Learning Basics**
    - Understanding supervised and unsupervised learning
    - Familiarity with algorithms like linear/logistic regression, KNN, and decision trees
    - Knowledge of model evaluation metrics
5. **Conceptual Foundations of Neural Networks**
    - Structure of perceptrons and activation functions
    - Loss functions and optimization techniques

    - Concepts like backpropagation and gradient descent
6. **Deep Learning Tools**
    - Basics of PyTorch or TensorFlow for building models

A clear understanding of these topics will significantly enhance your ability to grasp and implement neural network models effectively.

Introduction to Neural Networks

Neural networks are a class of machine learning models inspired by the structure and function of the human brain. They are at the core of modern artificial intelligence, powering applications in image recognition, natural language processing, speech synthesis, game playing, medical diagnosis, and more. This detailed introduction aims to provide an in-depth understanding of what neural networks are, how they work, and why they are powerful tools for solving complex problems.

https://www.datacamp.com/blog/what-are-neural-networks (continue reading here)

https://www.youtube.com/watch?v=MfIjxPh6Pys&t=6s&ab_channel=StanfordOnline (Harvard Lecture must watch)

---

1. What is a Neural Network?

A neural network is a computational system that learns patterns from data through interconnected layers of artificial neurons. Each neuron receives inputs, processes them, and passes the result to the next layer. The network adjusts its internal parameters (weights and biases) during training to minimize the error in predictions.

---

2. Biological Inspiration

Neural networks are inspired by the human brain, where billions of neurons are connected via synapses. In artificial neural networks (ANNs), each "neuron" is a mathematical function that computes a weighted sum of its inputs and applies a non-linear activation function. This biologically inspired design allows the model to learn complex mappings from input to output.

---

3. Basic Architecture

A typical neural network consists of three types of layers:

- **Input Layer**: Takes in the features of the dataset.
- **Hidden Layers**: Perform computations via neurons. More hidden layers enable the network to learn more complex representations.
- **Output Layer**: Produces the final prediction or classification.

Each neuron in a layer is connected to every neuron in the next layer (in a fully connected feedforward network).

---

4. Key Components

- **Weights and Biases**: Each connection between neurons has an associated weight. Bias allows the activation function to be shifted.

  https://milvus.io/ai-quick-reference/what-are-weights-and-biases-in-a-neural-network

- **Activation Function**: Introduces non-linearity into the model, enabling it to learn complex relationships. Examples include ReLU, Sigmoid, and Tanh.

  https://www.datacamp.com/tutorial/introduction-to-activation-functions-in-neural-networks

- **Loss Function**: Measures how far the prediction is from the actual result. Common loss functions include Mean Squared Error and Cross Entropy.

  https://towardsdatascience.com/loss-functions-and-their-use-in-neural-networks-a470e703f1e9/

- **Optimizer**: Algorithm that updates weights to minimize the loss function. Examples include SGD, Adam, and RMSprop.

  https://www.youtube.com/watch?v=LoGPU2oXp8g

---

5. Forward Propagation

During forward propagation, the input data passes through the layers of the network. Each layer computes its output using the weighted sum of inputs plus bias, followed by an activation function.

Mathematically:

$Z = W * X + b$

$A = \text{activation}(Z)$

Where:

- `W` is the weight matrix
- `X` is the input
- `b` is the bias
- `A` is the output after activation

https://www.datacamp.com/tutorial/forward-propagation-neural-networks (Learning Resource)

---

6. Backpropagation and Training

To train a neural network, the model makes predictions and compares them to the true labels using the loss function. Then, using **backpropagation**, the network calculates gradients of the loss with respect to weights and updates them using the optimizer.

https://youtu.be/1GnfvhBUs_E

https://youtu.be/6M1wWQmcUjQ

https://www.youtube.com/watch?v=ma6hWrU-LaI

This process involves:

- **Computing the error**
- **Calculating the gradient using chain rule (backpropagation)**
- **Updating the weights using gradient descent or a variant**

---

7. Types of Neural Networks

There are various types of neural networks suited for different tasks:

- **Feedforward Neural Networks (FNNs)**: Basic architecture where data moves in one direction.
  https://www.youtube.com/watch?v=AsyPA69QBks&ab_channel=ProfessorBryce
- **Convolutional Neural Networks (CNNs)**: Used for image processing and computer vision.

  https://www.youtube.com/watch?v=QzY57FaENXg&ab_channel=IBMTechnology

  https://www.youtube.com/watch?v=hDVFXf74P-U&t=776s&ab_channel=CampusX

- **Recurrent Neural Networks (RNNs)**: Designed for sequence data like time series or natural language.

  https://www.youtube.com/watch?v=Gafjk7_w1i8&ab_channel=IBMTechnology

  https://www.youtube.com/watch?v=BjWqCcbusMM&ab_channel=CampusX

  https://www.youtube.com/watch?v=LHXXI4-IEns&ab_channel=TheAIHacker

- **Long Short-Term Memory (LSTM)**: A type of RNN that handles long-term dependencies.

  https://www.youtube.com/watch?v=b61DPVFX03I&ab_channel=IBMTechnology

  https://www.youtube.com/watch?v=YCzL96nL7j0&ab_channel=StatQuestwithJoshStarmer

  https://www.youtube.com/watch?v=4KpRP-YUw6c&ab_channel=CampusX

- **Transformers**: Advanced architecture used in language models like GPT and BERT.
  https://www.youtube.com/watch?v=SZorAJ4I-sA&ab_channel=GoogleCloudTech

  https://www.youtube.com/watch?v=ZXiruGOCn9s&ab_channel=IBMTechnology

  https://www.youtube.com/watch?v=zxQyTK8quyY&t=1550s&ab_channel=StatQuestwithJoshStarmer

  https://www.youtube.com/watch?v=t45S_MwAcOw&ab_channel=GoogleCloudTech

  https://www.cloudskillsboost.google/course_templates/538 (Important)

  https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

---

8. Why Neural Networks Work

Neural networks can approximate any continuous function, given enough neurons and layers — this is known as the **Universal Approximation Theorem**. Their ability to learn representations from raw data, without manual feature engineering, makes them extremely powerful.

Attention mechanism: overview

https://www.youtube.com/watch?v=fjJOgb-E41w&ab_channel=GoogleCloudTech

---

9. Challenges in Neural Networks

- **Overfitting**: When a model performs well on training data but poorly on unseen data.

  https://www.youtube.com/watch?v=fUYMg23L_XI

- **Vanishing/Exploding Gradients**: When gradients become too small or large during backpropagation.

  https://www.youtube.com/watch?v=-J0P1PEIgLo

- **Training Time and Resources**: Deep networks require large amounts of data and computational power.

Solutions include:

- Regularization (dropout, L2)
- Batch normalization

- Better weight initialization
- Using modern optimizers like Adam

---

10. Modern Tools and Frameworks

Neural networks are built using deep learning frameworks such as:

- **TensorFlow**
- **PyTorch**
- **Keras**

These tools provide high-level APIs and automatic differentiation, simplifying the implementation and training of complex models.

---

11. Applications of Neural Networks

- **Computer Vision**: Object detection, facial recognition, self-driving cars
- **Natural Language Processing**: Translation, sentiment analysis, chatbots
- **Healthcare**: Disease detection, diagnostics, personalized medicine
- **Finance**: Fraud detection, stock market prediction
- **Robotics**: Motion control, perception, and decision-making

---

Final Words

Neural networks are a transformative technology that continues to evolve rapidly. Mastering their fundamentals enables one to explore the frontiers of artificial intelligence, solve real-world problems, and contribute to groundbreaking innovations.

This document is your gateway to understanding the foundation upon which deep learning is built. In the chapters ahead, we will delve deeper into the mechanisms, architectures, and applications of neural networks.

Top Learning Resources:

Distill.pub

- Explains machine learning concepts visually and intuitively.
- https://distill.pub/

fast.ai Practical Deep Learning for Coders

- Hands-on course focused on quickly building deep learning models.
- https://course.fast.ai/

3Blue1Brown's Neural Networks Series (YouTube)

- Visual and conceptual explanation of how neural networks work.
- https://www.youtube.com/playlist?list=PLZHQObOWTQDMsr9K-rj53DwVRMYO3t5Yr

Towards Data Science (Medium Blog)

- Community blog with beginner to advanced neural network tutorials.
- https://towardsdatascience.com

PyTorch Official Tutorials

- Hands-on guides for using PyTorch in deep learning.
- https://pytorch.org/tutorials/

Deep Learning with PyTorch by Eli Stevens, Luca Antiga, and Thomas Viehmann

- Focuses on practical applications using PyTorch.
- https://www.manning.com/books/deep-learning-with-pytorch